NUESTRA SEÑORA DE GUIA ACADEMY OF MARIKINA

Online Enrollment Management System

Technical Documentation

SYSTEM	NSDGA Enrollment (web application)
AUDIENCE	Developers / maintainers
FRONTEND	React 19 · Vite · Tailwind CSS v4 · daisyUI
BACKEND	Supabase (Auth · PostgreSQL · Storage · Edge Functions)
EMAIL	Brevo transactional API
HOSTING	Vercel (frontend) · Supabase (backend)
VERSION	1.0
DATE	September 2026

Confidential — internal engineering reference. Contains architecture and security details.

Table of Contents

1. Introduction & Overview 3
2. Technology Stack 4
3. System Architecture 5
4. Frontend Architecture 6
5. Backend: Database & Security 7
6. How the Frontend & Backend Communicate 9
7. API: Edge Functions 10
8. Key Feature Workflows 11
9. Security Model 13
10. Deployment & Environment 14
11. Local Setup & Migration Order 15
12. Database Schema Reference 16
13. Glossary 17

1. Introduction & Overview

The NSDGA Enrollment System is a single-page web application that digitizes admission and student-records management for Nuestra Señora de Guia Academy of Marikina. It lets prospective students apply online, staff review and enroll them, organize class sections, run the full student lifecycle (enroll !' complete !' re-enroll !' graduate/ withdraw/archive), hand off billing to the Finance office, and maintain the public website — all backed by a secure, role-aware Supabase backend.

User roles

Access is governed by four roles. Each role has a dedicated portal and route guard.

Role	Portal	Responsibilities
student	Student Portal	Apply online, upload documents, track status, view class & tuition, re-enroll.
registrar	Registrar Portal	Review applications, verify documents, enroll/reject/return, sections, lifecycle.
admin	Admin Portal	Website content, section approvals, Billing & Finance handoff.
headadmin	Head Admin Portal	Create/verify staff accounts, roles, audit log, presence, reports.

2. Technology Stack

Frontend

Technology	Role	Why it was chosen
React 19	UI library	Component model, hooks, large ecosystem; latest concurrent features.
Vite	Build tool / dev server	Instant HMR, fast production builds via Rollup/Rolldown.
React Router 7	Client-side routing	Declarative routes + nested layouts + route guards (SPA).
Tailwind CSS v4	Styling	Utility-first, responsive breakpoints (sm/md/lg) out of the box.
daisyUI	Component classes	Prebuilt UI primitives (modal, drawer, carousel) on top of Tailwind.
react-icons	Icons	Single dependency for the Hi/Lu icon sets used across portals.

Backend & Platform

Technology	Role	Why it was chosen
Supabase Auth	Authentication	Email/password + JWT sessions; MFA available; managed.
PostgreSQL (Supabase)	Database	Relational store with Row-Level Security enforced in the DB.
Supabase Storage	File storage	Private bucket for student documents + signed URLs.
Edge Functions (Deno)	Server-side logic	Privileged operations with the service-role key (TypeScript).
Brevo	Transactional email	Free tier (300/day), no domain required, simple REST API.
Vercel	Frontend hosting	Git-based deploys, global CDN, zero-config for Vite SPAs.
GitHub	Version control	Source of truth; Vercel deploys on push to main.

3. System Architecture

The application is a client-rendered SPA. The browser loads the React bundle from Vercel, then talks directly to Supabase over HTTPS using the supabase-js client. Every request carries the user's JWT; PostgreSQL Row-Level Security (RLS) decides what each request may read or write. Privileged actions that must bypass RLS (creating accounts, sending email) run inside Supabase Edge Functions using the service-role key — never in the browser.

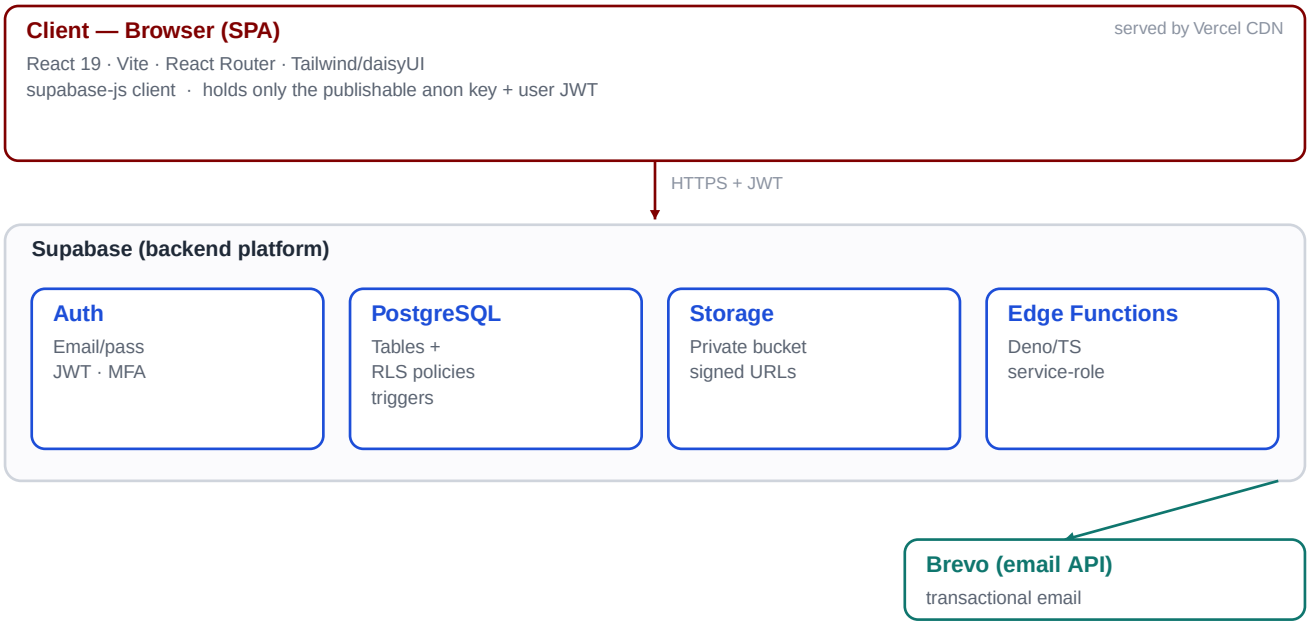

Figure 3.1 — High-level architecture. The browser never holds privileged keys; RLS and Edge Functions enforce all trust boundaries.

Key principle

- **Publishable anon key in the browser** is safe by design — it grants nothing on its own; RLS gates every row.
- **Service-role key never leaves the server** it lives only in Edge Function environment variables.
- **The database is the security boundary** not the UI. Hiding a button never grants or denies access; RLS does.

4. Frontend Architecture

Project structure

```
src/
  App.jsx                 Route table (all portals + guards)
  auth/authStore.js       Auth + audit data layer (signIn, recordAudit, accounts)
  contexts/               SessionProvider, SessionContext, useSession hook
  lib/                    Data-access layer (one module per domain)
    supabase.js           The single supabase-js client
    registrar.js          Applications, enrollment, sections, lifecycle
    student.js            Apply, my application, documents, re-enroll
    billing.js            Billing status + CSV export/import
    content.js            Public site content (announcements, contact, events)
    grades.js  print.js   Grade helpers, print-to-PDF class lists
  components/
    pages/<Portal>/       Screens per role (Student/Registrar/Admin/HeadAdmin)
    <Portal>Sidebar.jsx   Navigation shells (with mobile drawer)
```

Session & routing

SessionProvider loads the Supabase session on mount and merges the auth user with their profile row into one flat object (id, email, name, role, status). It exposes login/logout/refresh via React context (useSession). Routes are wrapped in a ProtectedRoute that checks the role and verification status before rendering.

```
<Route path="/AdminPortal/Billing" element={
  <ProtectedRoute allow={['admin','headadmin']}
                  loginPath="/AdminPortal/Login" requireVerified>
    <Billing />
  </ProtectedRoute>
} />
```

The data-layer pattern

UI components never call supabase directly. Each screen imports functions from lib/*.js, which own all queries. This keeps SQL/RLS concerns in one place and the components declarative.

```
// lib/student.js — the signed-in student's latest application
export async function getMyApplication() {
  const { data: u } = await supabase.auth.getUser();
  if (!u?.user) return null;
  const { data } = await supabase.from('applications').select('*')
    .eq('student_id', u.user.id)
    .order('created_at', { ascending: false }).limit(1).maybeSingle();
  return data;
}
```

5. Backend: Database & Security

All application data lives in PostgreSQL. Six core tables model the domain. Row-Level Security is enabled on every table, and database triggers enforce invariants that the UI cannot bypass.

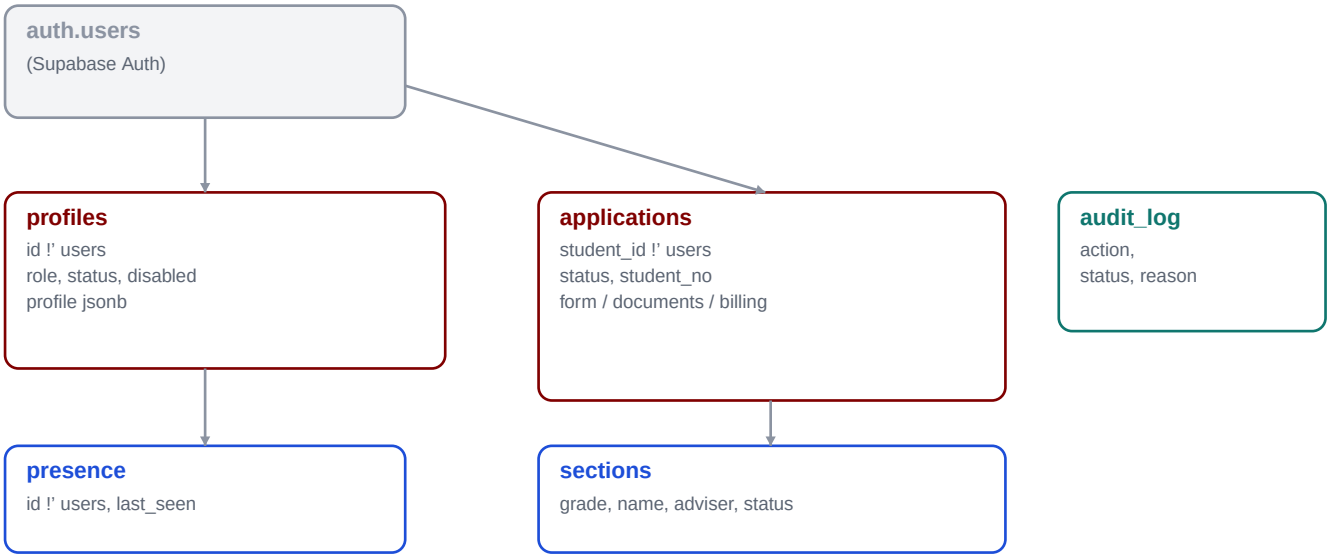

Figure 5.1 — Core tables. profiles and applications both key off auth.users; presence and profiles are 1:1 with a user.

The is_staff() helper & RLS

A SECURITY DEFINER function answers "is the caller verified staff?" without causing RLS recursion. Policies call it to grant staff full read/write while students see only their own rows.

```
create or replace function public.is_staff() returns boolean
language sql security definer set search_path = public stable as $$
  select exists (select 1 from public.profiles
    where id = auth.uid()
      and role in ('registrar','admin','headadmin')
      and status = 'verified');
$$;

-- applications: a student sees only their row; staff see all
create policy "app_select" on public.applications for select
  using (student_id = auth.uid() or public.is_staff());
```

Trigger-enforced invariants

RLS decides row visibility; triggers decide which fields a non-staff user may change. A student may edit their own application form, but cannot self-enroll, set a student number, or alter billing — the trigger raises an exception if they try.

```
create or replace function public.enforce_application_update()
returns trigger language plpgsql security definer as $$
begin
  if auth.uid() is null or public.is_staff() then return new; end if;
  if new.status is distinct from old.status
     and not (old.status='needs_correction' and new.status='submitted')
    then raise exception 'Not allowed to change application status'; end if;
  if new.student_no is distinct from old.student_no
    then raise exception 'Not allowed to set a student number'; end if;
  if new.billing is distinct from old.billing
    then raise exception 'Not allowed to change billing'; end if;
  return new;
end; $$;
```

Storage: private student documents

Sensitive documents (PSA, report cards) live in a PRIVATE bucket. Files are stored under a per-student folder ({user_id}/…) and are viewed only through short-lived (5-minute) signed URLs. Storage policies mirror the DB: a student touches only their own folder; staff read all.

```
-- upload path:  {user_id}/{doc}-{timestamp}.{ext}
create policy "student_docs_select" on storage.objects for select
  to authenticated using (
    bucket_id = 'student-docs'
    and ((storage.foldername(name))[1] = auth.uid()::text
         or public.is_staff()));
```

6. How the Frontend & Backend Communicate

There are three channels between the browser and Supabase, each with a different trust model.

- **1. PostgREST (auto REST) via supabase-js** CRUD on tables. The client attaches the user JWT; RLS authorizes each row. This is the default path for almost all reads/writes.
- **2. Storage API** Uploads to the private bucket and creation of signed URLs for viewing documents.
- **3. Edge Functions (RPC-style)** supabase.functions.invoke(name, { body }) for privileged operations that need the service-role key.

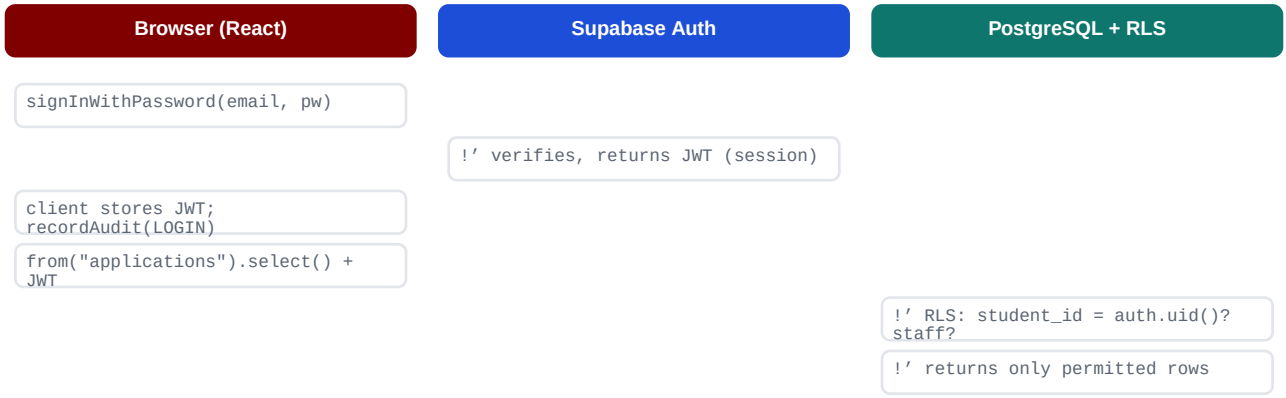

Figure 6.1 — Login + authorized read. The JWT rides on every request; RLS is the gatekeeper.

Example: an authorized write

When the Registrar enrolls a student, the UI calls a lib function, which issues an UPDATE. RLS confirms the caller is staff; the trigger allows staff to set status + student_no; then a fire-and-forget email is sent via the notify Edge Function.

```
// lib/registrar.js (abridged)
const { error } = await supabase.from('applications').update({
  status: 'enrolled', student_no, reviewed_by: user?.id,
  reviewed_at: new Date().toISOString(), review_note: msg,
}).eq('id', id);
logAction('ENROLL', `Enrolled ${grade} · ${name} (${student_no})`);
notifyStudent(app.student_id, subject, html);   // -> notify Edge Function
```

7. API: Edge Functions

Four Deno/TypeScript functions run server-side. Each sets CORS, validates the caller, and uses the service-role key only after authorization. They are invoked from the client with supabase.functions.invoke().

create-applicant (public)

- **Purpose** Create a confirmed student account + file their admission application, atomically.
- **Auth** Public, but can ONLY ever create the lowest-privilege student role.
- **In** { email, password, contact, level, studentType, form } !' Out: { ok, userId }
- **Safety** If the application insert fails, the just-created user is deleted (rollback).

```
const { data: created } = await admin.auth.admin.createUser({
  email: clean, password, email_confirm: true,
  user_metadata: { name, role: 'student', contact },
});
const { error: appErr } = await admin.from('applications').insert({
  student_id: created.user.id, school_year: form.schoolYear,
  grade_level: level, student_type: studentType,
  status: 'submitted', form: { ...form, contact },
});
if (appErr) { await admin.auth.admin.deleteUser(created.user.id); }
```

create-staff (HeadAdmin only)

- **Purpose** Create a registrar/admin account without sending a confirmation email.
- **Auth** Reads the caller JWT, then requires role === headadmin before proceeding.
- **In** { name, email, password, role } !' Out: { id, email, name, role }

notify (staff only) !' Brevo

- **Purpose** Send a transactional email (enroll/correction/reject) to a student.
- **Privacy** The recipient email is resolved server-side from studentId — never exposed to the client.
- **In** { studentId, subject, html } !' Brevo POST /v3/smtp/email

```
await fetch('https://api.brevo.com/v3/smtp/email', {
  method: 'POST',
  headers: { 'api-key': brevoKey, 'Content-Type': 'application/json' },
  body: JSON.stringify({
    sender: { name: fromName, email: fromEmail },
    to: [{ email: to }], subject, htmlContent: html,
  }),
});
```

seed-test-data (HeadAdmin only — dev utility)

Wipes all student accounts and seeds fresh test students. This is a TEST utility and MUST be deleted before production (see §9).

8. Key Feature Workflows

8.1 Online admission

- Applicant fills the multi-step wizard (grade/type !’ account !’ student info !’ guardian !’ academic !’ review).
- Submit !’ create-applicant makes a confirmed student account + application (status = submitted).
- Registrar reviews !’ Enroll (assigns student number), Return for correction, or Reject — each emails the student.

8.2 Document verification

Students upload the required documents into the private bucket; the file map is stored on applications.documents. The Registrar opens each via a signed URL and marks it verified or needs re-upload.

8.3 Class sections & approval

- Registrar creates sections per grade/year and assigns enrolled students (GWA-sorted); prints class lists.
- Admin approves a section !’ the roster (adviser + classmates) is written onto each member's own record, so students see their class without ever reading other students' rows.

8.4 Student lifecycle

Records are never deleted. Statuses model the full lifecycle; deprovisioned students lose login (reversible).

Status	Meaning
`submitted / under_review`	Application filed / being reviewed.
`needs_correction`	Sent back to the student to fix and resubmit.
`enrolled`	Accepted; has a student number and portal access.
`completed`	Finished a school year; eligible to re-enroll into the next grade.
`rejected`	Not accepted.
`withdrawn / graduated / archived`	Deprovisioned — login revoked, record kept (reversible).

8.5 Billing & Finance handoff (voucher-aware)

Finance is handled off-system. The Admin Billing page EXPORTS enrolled students to CSV, Finance assesses/collects offline, then the Admin IMPORTS the returned CSV — matched to students by Student No — to update each record's billing status. SHS applicants can flag a DepEd voucher (QVR/Non-QVR) in the wizard; students see a read-only Tuition & Payment card.

```
// lib/billing.js — import matches rows by Student No, then updates billing
const byNo = {}; rows.forEach(r => { if (r.student_no) byNo[r.student_no] = r; });
for (const line of parsedCsv) {
  const app = byNo[line['Student No']];
  if (app) await updateBilling(app.id, { status, amountDue, amountPaid,
    orNumber, remarks }, applicantName(app));
}
```

8.6 Audit log & presence

Every login/logout and every staff action (enroll, reject, lifecycle, section approval, account changes, content edits, billing updates) is written to audit_log via logAction(). A quiet heartbeat upserts presence.last_seen every ~90s so Head Admin sees who is online.

```
// contexts/SessionProvider.jsx — presence heartbeat
useEffect(() => {
  const uid = session?.id; if (!uid) return;
  const beat = () => supabase.from('presence')
    .upsert({ id: uid, last_seen: new Date().toISOString() });
  beat(); const iv = setInterval(beat, 90000);
  return () => clearInterval(iv);
}, [session?.id]);
```

9. Security Model

- **Defense in the database** RLS + SECURITY DEFINER triggers enforce access and field-level invariants regardless of the client.
- **Least privilege** The public applicant endpoint can only create student accounts; staff creation requires a HeadAdmin JWT.
- **Private documents** Never public — only per-owner access + short-lived signed URLs.
- **Audit trail** Login/logout and staff actions are recorded; the audit table can be hardened so users may only insert rows about themselves (harden_audit.sql).
- **Data privacy** PII is collected under the Data Privacy Act 2012 (consent captured in the wizard).

Must be removed / changed before production

- **Delete seed-test-data** It can wipe and reseed student data; it is a dev-only utility.
- **Drop demo_password** A dev-only plaintext column: alter table public.profiles drop column demo_password;
- **Re-enable MFA** MFA_REQUIRED_ROLES is currently empty; enable it for staff and remove the demo bypass.
- **Custom SMTP + backups** Move off shared email limits; enable database backups; use a dedicated production project.

10. Deployment & Environment

Frontend (Vercel)

- Push to the main branch !’ Vercel builds (vite build) and deploys to the CDN.
- The git commit author must match the linked Vercel account for Hobby-tier deploys.
- Environment variables (Project !’ Settings): VITE_SUPABASE_URL, VITE_SUPABASE_ANON_KEY.

Backend (Supabase)

- Run the SQL migrations in the SQL Editor (order in §11).
- Deploy Edge Functions: supabase functions deploy <name>.
- Function secrets: BREVO_API_KEY, NOTIFY_FROM_EMAIL, NOTIFY_FROM_NAME. (SUPABASE_URL / keys are injected automatically.)

```
# .env.local (frontend, git-ignored)
VITE_SUPABASE_URL=https://<project>.supabase.co
VITE_SUPABASE_ANON_KEY=<publishable anon key>
```

11. Local Setup & Migration Order

```
git clone <repo> && cd NSDGA-enrollment
npm install
# create .env.local with the two VITE_ vars above
npm run dev        # http://localhost:5173
npm run build      # production build
npm run lint       # ESLint
```

SQL migration order (Supabase SQL Editor)

#	File	Adds
1	`schema.sql`	profiles, audit_log, handle_new_user, is_headadmin, RLS.
2	`students.sql`	applications table, is_staff(), enforce_application_update, RLS.
3	`student-docs.sql`	private student-docs storage bucket + policies.
4	`lifecycle.sql / lifecycle2.sql`	withdrawn/graduated/archived + completed statuses.
5	`readmit.sql`	staff may insert applications (re-admission).
6	`sections.sql / sections2.sql`	sections table + approval status.
7	`presence.sql`	presence table (last_seen) + policies.
8	`billing.sql`	applications.billing column + trigger guard.
9	`harden_audit.sql`	(optional) restrict audit inserts to self.

12. Database Schema Reference

profiles — one row per auth user

Column	Type	Notes
id	uuid PK	FK !' auth.users (cascade delete)
email / name	text	Copied from auth on signup
role	text	registrar | admin | headadmin | student
status	text	pending_profile | pending_verification | verified
disabled	boolean	Login blocked when true
profile	jsonb	Staff profile details
created_at / verified_at	timestamptz	Lifecycle timestamps

applications — one row per student per school year

Column	Type	Notes
id	uuid PK	gen_random_uuid()
student_id	uuid	FK !' auth.users (cascade)
school_year / grade_level	text	e.g. 2026-2027 / Senior High School
student_type	text	New / Transferee / Returnee
status	text	submitted … enrolled … graduated (see §8.4)
form	jsonb	All wizard fields (name, grade, strand, voucher…)
documents	jsonb	Uploaded file map (path, status)
billing	jsonb	Payment status (staff-managed)
student_no	text	Assigned by registrar on enroll
reviewed_by / reviewed_at	uuid / ts	Audit of the decision

sections / audit_log / presence / site_content

Column	Type	Notes
sections	table	id, school_year, grade, name, adviser, capacity, status
audit_log	table	id, created_at, email, name, role, action, status, reason, ip, device
presence	table	id (PK!'user), last_seen
site_content	table	section (PK), content jsonb, updated_at, updated_by

13. Glossary

Term	Meaning
RLS	Row-Level Security — PostgreSQL policies that authorize access per row.
JWT	JSON Web Token — the signed session token supabase-js sends on each request.
anon key	Publishable client key; safe in the browser because RLS gates everything.
service-role key	Full-access key used only inside Edge Functions (never in the browser).
SECURITY DEFINER	A function that runs with its owner's rights, used to avoid RLS recursion.
Edge Function	A Deno/TypeScript function hosted by Supabase for privileged server logic.
Signed URL	A time-limited link that grants temporary read access to a private file.
QVR	Qualified Voucher Recipient — a DepEd SHS voucher category.